

Lecture 6: Synchronization

601.418/618 Operating Systems

David Hovemeyer

February 10, 2024

Agenda

- ▶ Why synchronization is necessary
- ▶ Critical sections and mutual exclusion
- ▶ Implementing synchronization

Acknowledgments: These slides are shamelessly adapted from [Prof. Ryan Huang's Fall 2022 slides](#), which in turn are based on [Prof. David Mazières's OS lecture notes](#).

Before we start: Too Much Milk

Time	Alice	Bob
12:30	Look in fridge. Out of milk.	
12:35	Leave for store.	
12:40	Arrive at store.	Look in fridge. Out of milk.
12:45	Buy milk.	Leave for store.
12:50	Arrive home, put milk away.	Arrive at store.
12:55		Buy milk.
1:00		Arrive at home, put milk away. Oh no!

Before we start: Exercise # 1

// x is a global variable initialized to 0

// Thread 1

```
void foo()  
{  
    x++;  
}
```

// Thread 2

```
void bar()  
{  
    x--;  
}
```

After both threads finish, what is the value of x?

- ▶ It could be 0, 1, or -1
- ▶ Why?

Before we start: Exercise # 2

```
int p = 0, ready = 0;
```

```
// processor #1
```

```
p = 1000;
```

```
ready = 1;
```

```
// processor #2
```

```
while (!ready) /* do nothing */;
```

```
use(p);
```

What value of p is passed to use()?

- ▶ It could be 0 or 1000?
- ▶ Why?

What if p holds an address?

Synchronization: motivation

Threads cooperate in multithreaded programs

- ▶ *To share resources*, access shared data structures
- ▶ *To coordinate* their execution

For correctness, we need to control this cooperation

- ▶ Thread schedule is **non-deterministic** (i.e., behavior could be different when we re-run the program)
 - ▶ Scheduling is not under program control
 - ▶ Threads *interleave executions arbitrarily* and at different rates
- ▶ Multi-word operations are not atomic
- ▶ Compiler/hardware instruction reordering

Shared resources

We initially focus on controlling access to shared resources

Basic problem

- ▶ If two concurrent threads (processes) are accessing a shared variable, and that variable is read/modified/written by those threads, then access to the variable must be controlled to avoid erroneous behavior

Over the next couple of lectures, we will look at

- ▶ *Mechanisms to control access to shared resources*
 - ▶ Locks, mutexes, semaphores, monitors, condition variables, etc.
- ▶ *Patterns for coordinating accesses to shared resources*
 - ▶ Bounded buffer, producer-consumer, etc.

Classic example: bank account balance

Implement a function to handle withdrawals from a bank account:

```
Money withdraw(Account *account, Money amount) {  
    Money balance;  
    balance = get_balance(account);  
    balance = balance - amount;  
    put_balance(account, balance);  
    return balance;  
}
```

Suppose that you and your significant other share a bank account with a balance of \$1000

Then you each go to separate ATM machines and simultaneously withdraw \$100 from the account

Classic example: bank account balance

We'll represent the situation by creating a separate thread for each person to do the withdrawals

These threads run on the same bank server:

// Thread 1

```
Money withdraw(Account *account, Money amount) {  
    Money balance;  
    balance = get_balance(account);  
    balance = balance - amount;  
    put_balance(account, balance);  
    return balance;  
}
```

// Thread 2

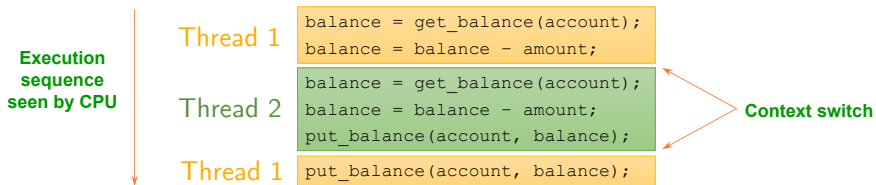
```
Money withdraw(Account *account, Money amount) {  
    Money balance;  
    balance = get_balance(account);  
    balance = balance - amount;  
    put_balance(account, balance);  
    return balance;  
}
```

What's the problem with this implementation?

- Think about potential schedules of these two threads

Interleaved Schedules

The problem is that the execution of the two threads can be interleaved:



What is the balance of the account now?

Is the bank happy with our implementation?

How interleaved can it get?

How contorted can the interleavings be?

We'll assume that the only atomic operations are instructions

- ▶ E.g., reads and writes of words
- ▶ The hardware may not even give you that!

We'll assume that a context switch can occur at any time

We'll assume that you can delay a thread as long as you like as long as it's not delayed forever

```
..... get_balance(account);
```

```
balance = get_balance(account);
```

```
balance = .....
```

```
balance = balance - amount;
```

```
balance = balance - amount;
```

```
put_balance(account, balance);
```

```
put_balance(account, balance);
```

Shared resources

Problem: concurrent threads accessed a *shared resource* without any *synchronization*

- ▶ Known as a *race condition*

We need mechanisms to control access to these shared resources in the face of concurrency

- ▶ So we can reason about how the program will operate

Our example was updating a shared bank account

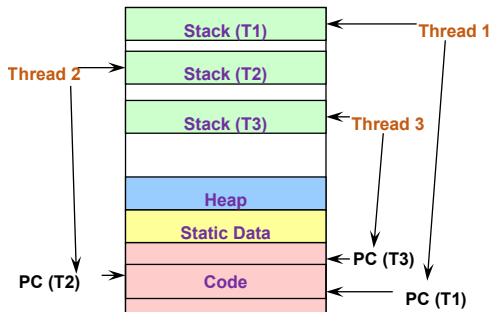
Synchronization is required for **any shared data structure**

- ▶ Buffers, queues, lists, hash tables, etc.
- ▶ Important exception: access to a read-only data structure doesn't require synchronization

When are resources shared?

Local variables are **not shared** (private)

- ▶ Refer to data on the stack
- ▶ Each thread has its own stack
- ▶ Never pass/share/store a pointer to a local variable on the stack for thread T1 to another thread T2



Global variables and static objects are **shared**

- ▶ Stored in the static data segment, accessible by any thread

Dynamic objects and other heap objects are **shared**

- ▶ Allocated from heap with malloc/free or new/delete

Mutual exclusion

We want to use *mutual exclusion* to synchronize access to shared resources

- ▶ This allows us to have larger atomic blocks

Code that uses mutual exclusion to synchronize its execution is called a *critical section*

- ▶ Only one thread at a time can execute in the critical section
- ▶ All other threads are forced to wait on entry
- ▶ When a thread leaves a critical section, another can enter
- ▶ Example: sharing your bathroom with housemates

What requirements would you place on a critical section?

Critical section requirements

1) Mutual exclusion (mutex)

- ▶ If one thread is in the critical section, then no other is

2) Progress

- ▶ If some thread T is not in the critical section, then T cannot prevent some other thread S from entering the critical section
- ▶ A thread in the critical section will eventually leave it

3) Bounded waiting (no starvation)

- ▶ If some thread T is waiting on the critical section, then T will eventually enter the critical section

4) Performance

- ▶ The overhead of entering and exiting the critical section is small with respect to the work being done within it

About requirements

There are three kinds of requirements that we'll use

Safety property: nothing bad happens

- ▶ Mutex

Liveness property: something good happens

- ▶ Progress, Bounded Waiting

Performance requirement

- ▶ Performance

Properties hold for *each run*, while performance depends on *all the runs*

- ▶ Rule of thumb: When designing a concurrent algorithm, worry about safety first (but don't forget liveness!)

Too Much Milk, Try #1

Try #1: leave a note

```
if (milk == 0) {    // if no milk
    if (note == 0) { // if no note
        note = 1;    // leave note
        milk++;      // buy milk
        note = 0;    // remove note
    }
}
```

What can go wrong?

Too Much Milk, Try #1

Try #1: leave a note

```
// Alice
```

```
if (milk == 0) {
```

```
    if (note == 0) {
```

```
        note = 1;
```

```
        milk++;
```

```
        note = 0;
```

```
    }
```

```
}
```

```
// Bob
```

```
if (milk == 0) {
```

```
    if (note == 0) {
```

```
        note = 1;
```

```
        milk++;
```

```
        note = 0;
```

```
    }
```

```
}
```

Too Much Milk, Try #2

Try #2: leave two notes

```
// Alice  
noteA = 1;  
if (noteB == 0) {  
    if (milk == 0) {  
        milk++;  
    }  
}  
noteA = 0;
```

```
// Bob  
noteB = 1;  
if (noteA == 0) {  
    if (milk == 0) {  
        milk++;  
    }  
}  
noteB = 0;
```

Is this safe?

Does it ensure liveness?

Mechanisms For Building Critical Sections

Atomic read/write

- ▶ Can it be done?

Locks

- ▶ Primitive, minimal semantics, used to build others

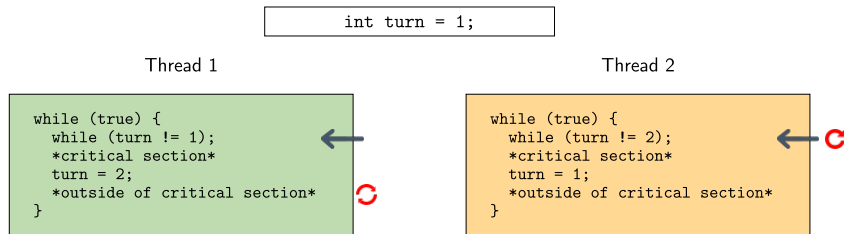
Semaphores

- ▶ Basic, easy to get the hang of, but hard to program with

Monitors

- ▶ High-level, requires language support, operations implicit

Mutex with Atomic R/W: Try #1



This is called *alternation*

Does it satisfy the safety requirement?

► Yes

Does it satisfy the liveness requirement?

► No, T1 can go into infinite loop outside of the critical section preventing T2 from entering

Mutex with Atomic R/W: Peterson's Algorithm

```
int turn = 1;  
bool try1 = false, try2 = false;
```

```
while (true) {  
    try1 = true;  
    turn = 2;  
    while (try2 && turn != 1) ;  
    critical section  
    try1 = false;  
    outside of critical section  
}
```

```
while (true) {  
    try2 = true;  
    turn = 1;  
    while (try1 && turn != 2) ;  
    critical section  
    try2 = false;  
    outside of critical section  
}
```

Does it satisfy the safety requirement?

Does it satisfy the liveness requirement?

Peterson's Algorithm

Intuitively: if a thread enters its critical section, it means either that

1. The other thread isn't trying to enter its critical section (e.g., thread 1 entered because `try2` is false), or
2. Both threads are trying to enter, but based on the value of `turn`, one thread "won" the race
 - ▶ The other thread will be allowed to enter when the "winner" ends its critical section

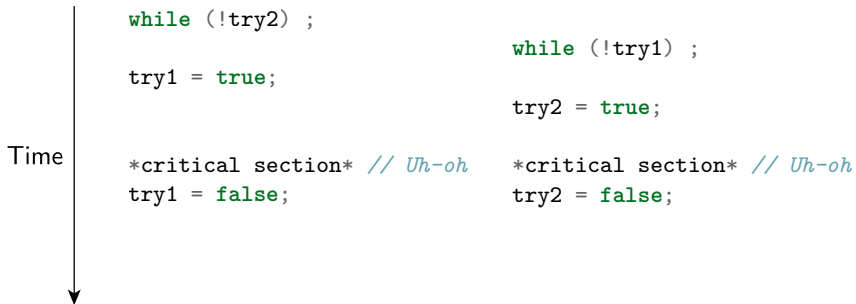
Peterson's Algorithm: Safety

The `turn` variable ensures safety: for mutual exclusion to be violated, it would necessary for both threads to have either

1. Observed the other thread's flag to be false (wouldn't happen if both threads are trying to enter their CS), or
2. Observed that `turn` was not set to the other thread's id (this would imply that $\text{turn} = 1 \wedge \text{turn} = 2$ if both threads are in their critical sections at the same time)

Peterson's Algorithm: Safety

If turn were not used (assume both try1 and try2 initially false):

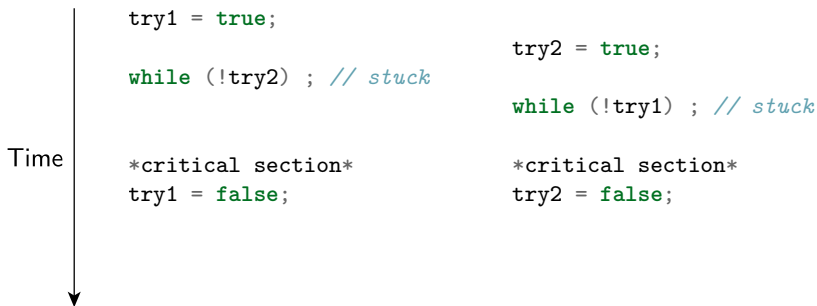


Peterson's Algorithm: Liveness

The “turn” variable is also needed to ensure liveness.

If we try to fix the unsafe version by assigning to `try1` and `try2` before waiting, there is a race where each thread sets its flag to true at about the same time, at which point no progress can be made (deadlock.)

Assuming both `try1` and `try2` initially false:



Peterson's Algorithm: Formal Proof

Peterson's Algorithm can be formally proven to be a correct solution which ensures safety:

- ▶ E.g., Dijkstra, [EWD779](#)

Should we use Peterson's algorithm?

From [the textbook](#) (Chapter 28):

For some reason, developing locks that work without special hardware support became all the rage for a while, giving theory-types a lot of problems to work on. Of course, this line of work became quite useless when people realized it is much easier to assume a little hardware support (and indeed that support had been around from the earliest days of multiprocessing). Further, algorithms like the ones above don't work on modern hardware (due to relaxed memory consistency models), thus making them even less useful than they were before. Yet more research relegated to the dustbin of history. . .

Locks

A lock is an object in memory providing two operations

- ▶ `acquire()`: wait until lock is free, then take it to enter a critical section
- ▶ `release()`: release lock to leave a critical section, waking up anyone waiting for it

Threads *pair calls* to acquire and release

- ▶ Between `acquire/release`, the thread *holds* the lock
- ▶ `acquire` does not return until any previous holder releases
- ▶ What can happen if the calls are not paired?

Locks can spin (a spinlock) or block (a mutex)

- ▶ Can break apart Peterson's to implement a spinlock

Too Much Milk, Try #4

// Alice

```
lock.acquire();  
if (milk == 0) {  
    milk++;  
}  
lock.release();
```

// Bob

```
lock.acquire();  
if (milk == 0) {  
    milk++;  
}  
lock.release();
```

Using locks

```
withdraw (account, amount) {  
    acquire(lock) ;  
    balance = get_balance(account);  
    balance = balance - amount;  
    put_balance(account, balance);  
    release(lock) ;  
    return balance;  
}
```

**Critical
Section**

```
acquire(lock) ;  
balance = get_balance(account);  
balance = balance - amount;
```

```
acquire(lock) ;  
  
put_balance(account, balance);  
release(lock) ;
```

```
balance = get_balance(account);  
balance = balance - amount;  
put_balance(account, balance);  
release(lock) ;
```

- ▶ What happens when green tries to acquire the lock?
- ▶ Why is the “return” outside the critical section? Is this ok?
- ▶ What happens when a third thread calls acquire?

Implementing locks (1)

How do we implement locks? Here is one attempt:

```
struct Lock {  
    int held = 0;  
};  
  
void acquire(struct Lock *lock) {  
    while (lock->held); // busy-wait for lock to be released  
    lock->held = 1;  
}  
  
void release(struct Lock *lock) {  
    lock->held = 0;  
}
```

This is called a *spinlock* because a thread spins waiting for the lock to be released

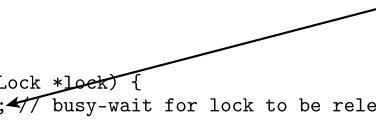
Does this work?

Implementing Locks (2)

No. Two independent threads may both notice that a lock has been released and thereby acquire it.

```
struct Lock {  
    int held = 0;  
};  
  
void acquire(struct Lock *lock) {  
    while (lock->held); // busy-wait for lock to be released  
    lock->held = 1;  
}  
  
void release(struct Lock *lock) {  
    lock->held = 0;  
}
```

Context switch could
occur here: causing a
race condition



Implementing Locks (3)

The problem is that the implementation of locks has critical sections, too!

How do we stop the recursion?

The implementation of acquire/release must be *atomic*

- ▶ An atomic operation is one which executes as though it could not be interrupted
- ▶ Code that executes “all or nothing”

How do we make them atomic?

Need help from hardware

- ▶ Atomic instructions (e.g., test-and-set)
- ▶ Disable/enable interrupts (prevents context switches)

Atomic Instructions: Test-And-Set

The semantics of test-and-set are:

- ▶ Record the old value
- ▶ Set the value to indicate available
- ▶ Return the old value

Hardware executes it atomically!

When executing test-and-set on “flag”

- ▶ What is **value of flag** afterwards if it was initially False? True?
- ▶ What is the **return result** if flag was initially False? True?

Other similar flavor atomic instructions: xchg, CAS

```
// Semantics:
```

```
bool test_and_set(bool *flag) {  
    atomically {  
        bool old = *flag;  
        *flag = True;  
        return old;  
    }  
}
```

Using Test-And-Set

Here is our lock implementation with test-and-set:

```
struct Lock {  
    int held = 0;  
};  
  
void acquire(struct Lock *lock) {  
    while (test_and_set(&lock->held));  
}  
  
void release(struct Lock *lock) {  
    lock->held = 0;  
}
```

- ▶ When will the while return? What is the value of held?
- ▶ What about multiprocessors?
- ▶ Implement it with xchg, CAS

Problems with Spinlocks

The problem with spinlocks is that they are wasteful

- ▶ If a thread is spinning on a lock, then the thread holding the lock cannot make progress (on a uniprocessor)

How did the lock holder give up the CPU in the first place?

- ▶ Lock holder calls `yield()` or `sleep()`
- ▶ Involuntary context switch

Only want to use spinlocks as primitives to build higher-level synchronization constructs

Disabling Interrupts

Another implementation of acquire/release is to disable interrupts:

```
struct lock {  
}  
  
void acquire(lock) {  
    disable_interrupts();  
}  
  
void release(lock) {  
    enable_interrupts();  
}
```

Note that there is no state associated with the lock

Can two threads disable interrupts simultaneously?

On Disabling Interrupts

Disabling interrupts blocks notification of external events that could trigger a context switch (e.g., timer)

- ▶ This is what Pintos uses as its primitive

In a “real” system, this is only available to the kernel

- ▶ Why?

Disabling interrupts is insufficient on a multiprocessor

- ▶ Interrupts are only disabled on a per-core basis
- ▶ Back to atomic instructions

Like spinlocks, only want to disable interrupts to implement higher-level synchronization primitives

- ▶ Don't want interrupts disabled between acquire and release

Summarize Where We Are

Goal: Use *mutual exclusion* to protect *critical sections* of code that access *shared resources*

Method: Use locks (either spinlocks or disable interrupts)

Problem: Critical sections (CS) can be long

Spinlocks

- ‡ Threads waiting to acquire lock spin in test-and-set loop
- ‡ Wastes CPU cycles
- ‡ Longer the CS, the longer the spin, greater the chance for lock holder to be interrupted

acquire(lock)

...

Critical section

...

release(lock)

Disabling Interrupts:

- ‡ Disabling interrupts for long periods of time can miss or delay important events (e.g., timer, I/O)

Higher-Level Synchronization

Spinlocks and disabling interrupts are useful only for very short and simple critical sections

- ▶ Wasteful otherwise
- ▶ These primitives are “primitive” – don’t do anything besides mutual exclusion

Need higher-level synchronization primitives that:

- ▶ **Block waiters**
- ▶ **Leave interrupts enabled** within the critical section

All synchronization requires atomicity

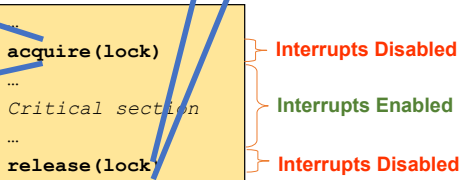
So we’ll use our “atomic” locks as primitives to implement them

Implementing locks (4)

Block waiters, interrupts enabled in critical sections

```
struct lock {  
    int held = 0;  
    queue Q;  
}  
  
void acquire(lock) {  
    Disable interrupts;  
    while (lock->held) {  
        put current thread on lock Q;  
        block current thread;  
    }  
    lock->held = 1;  
    Enable interrupts;  
}
```

```
void release(lock) {  
    Disable interrupts;  
    if (Q) remove waiting thread;  
    unblock waiting thread;  
    lock->held = 0;  
    Enable interrupts;  
}
```



See Pintos threads/synch.c: sema_down/up

Summary

Why we need synchronization

Critical sections

Simple algorithms to implement critical sections

Locks

Lock implementations

Next time

Semaphores and Monitors